언리얼 엔진 스터디 4주차

PlayerController · PlayerState · Enhanced Input · UMG

우리가 만드는 것 — Vampire Survivors-like

- **게임 컨셉**
  - 플레이어 시점 고정 / W·A·S·D 이동
  - 특수 능력을 키보드 단축키로 사용
  - 몰려오는 적과 HP 시스템 — 0이 되면 게임 오버
  - 플레이어 스탯은 죽어도 보존됨
- **로드맵**
  - **이번 주:** 스프린트 · HP 바 UI · 데미지 처리
  - **다음 주(5주차):** 적 스폰 시스템 + 피격 연출
  - **이후(6–8주차):** 경험치 · 레벨업 · 아이템 드롭
- **핵심 원칙:** 매주 실행 가능한 빌드 유지

우리가 만드는 것 — Vampire Survivors-like

- **이번 주 추가 기능**
  - 스프린트 기능
  - HP 스탯
  - HP 바 UI 표시

다른 기능들은 강의를 진행하며 차차 추가할 예정!

Overview

- **에디터 기초 숙달**
  - 뷰포트 조작 / 기즈모 / 플레이 모드
- **이론 A — PlayerController**
  - 실습 0 — Possess 체험
- **이론 B — PlayerState**
- **이론 C — Enhanced Input**
- **실습 1: 달리기 구현**
- **실습 2: HP 시스템 + HP 바 위젯 연동**
- **실습 3: 1초마다 데미지 + 리스폰**

오늘의 목표

- 실습 0: C++ PlayerController로 원하는 폰에 직접 빙의할 수 있다
- PlayerController / PlayerState의 역할과 차이를 설명할 수 있다
- Enhanced Input에서 InputMappingContext · InputAction을 연결할 수 있다
- 스프린트 기능을 C++로 구현하고 블루프린트에서 확인할 수 있다
- UMG로 간단한 HP 바 위젯을 만들고 화면에 표시할 수 있다
- PlayerState 기반으로 1초마다 데미지를 처리하고, 죽으면 리스폰시킬 수 있다

에디터 기초 숙달

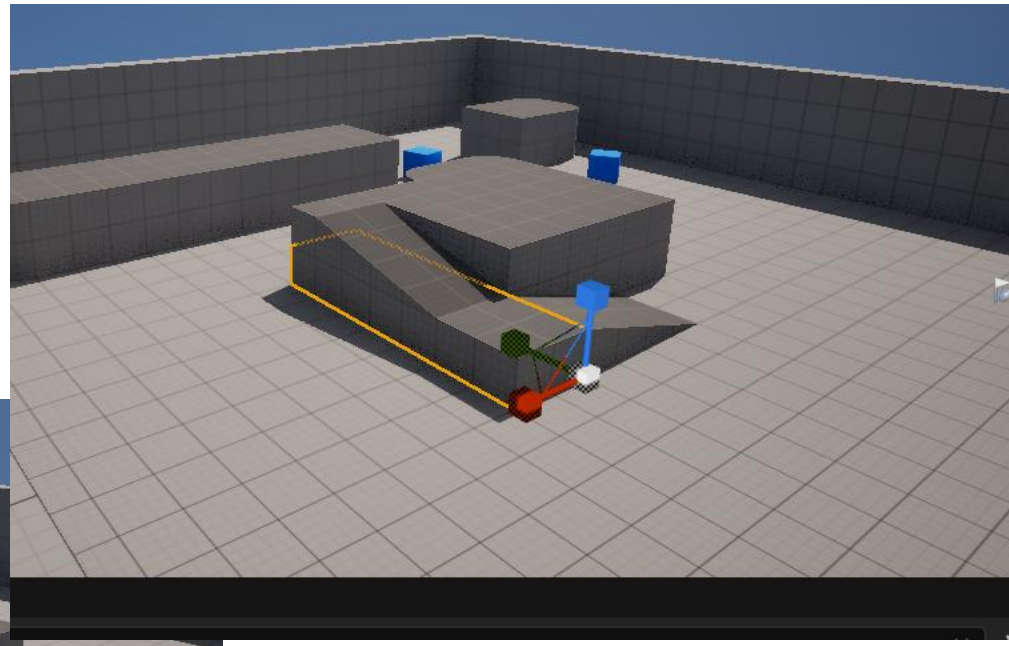
뷰포트 · 기즈모 · 블루프린트 에디터

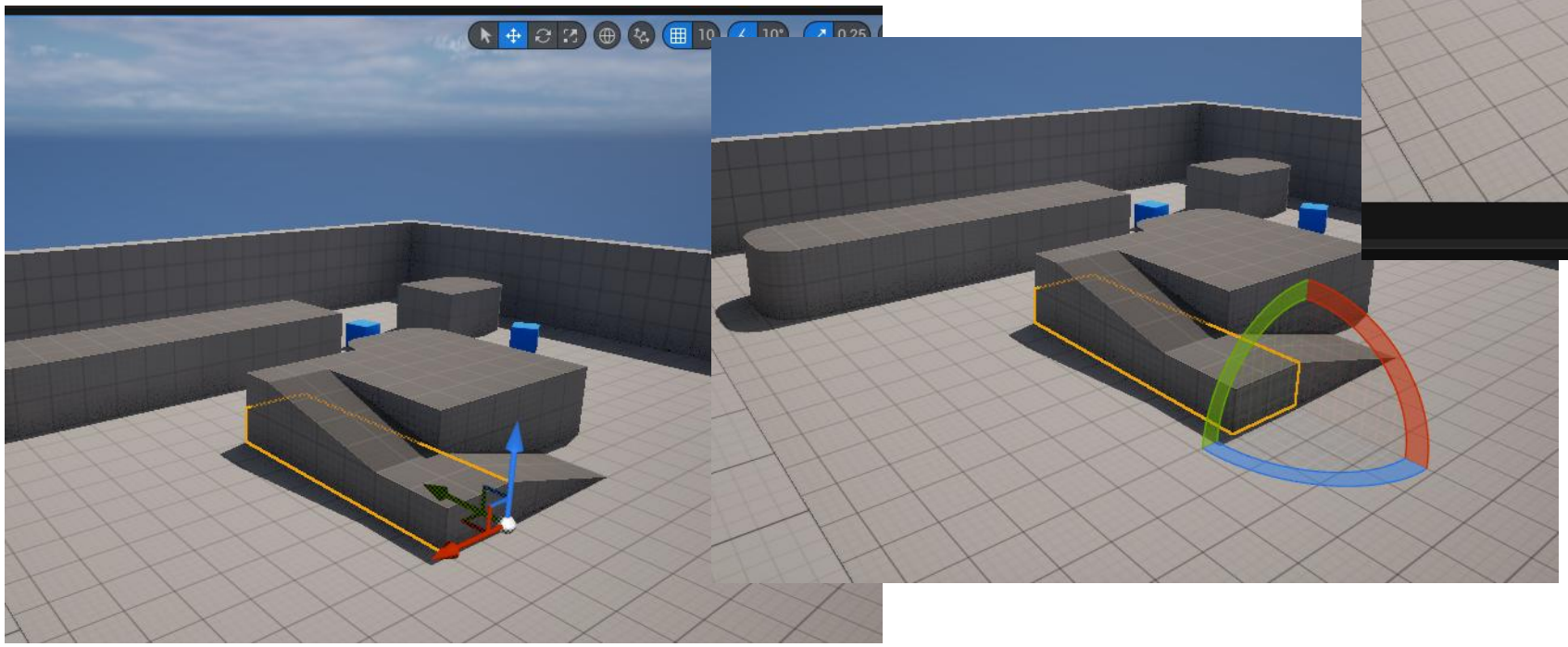
뷰포트 조작 — 마우스 단축키

- 우클릭 드래그 → 카메라 회전 (FPS 시점)
- 우클릭 + W/A/S/D → 1인칭 이동
- Alt + 좌클릭 드래그 → 해당 클릭 좌표 기준으로 둘러보기
- Alt + 우클릭 드래그 → 줌인/아웃
- F → 선택 오브젝트 포커스
- End → 바닥에 붙이기 (Snap to Floor)

기즈모와 스냅

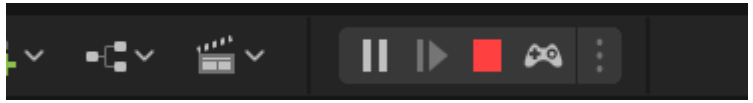
- W / E / R → 이동 · 회전 · 스케일 전환
- Space → 로컬/월드 좌표계 전환
- Ctrl+Z / Ctrl+Y → 실행 취소 / 재실행





플레이

- Play (뷰포트 안) — Alt+P
  - 현재 뷰포트에서 즉시 플레이. 가장 빠름
- Play (새 창) — 별도 창에서 실행
  - 해상도 테스트 / 멀티플레이 시뮬레이션에 유용
- Simulate — 플레이어 없이 월드만 시뮬레이션
  - AI·물리 동작 확인, 플레이어 컨트롤 없음



에디터 레이아웃

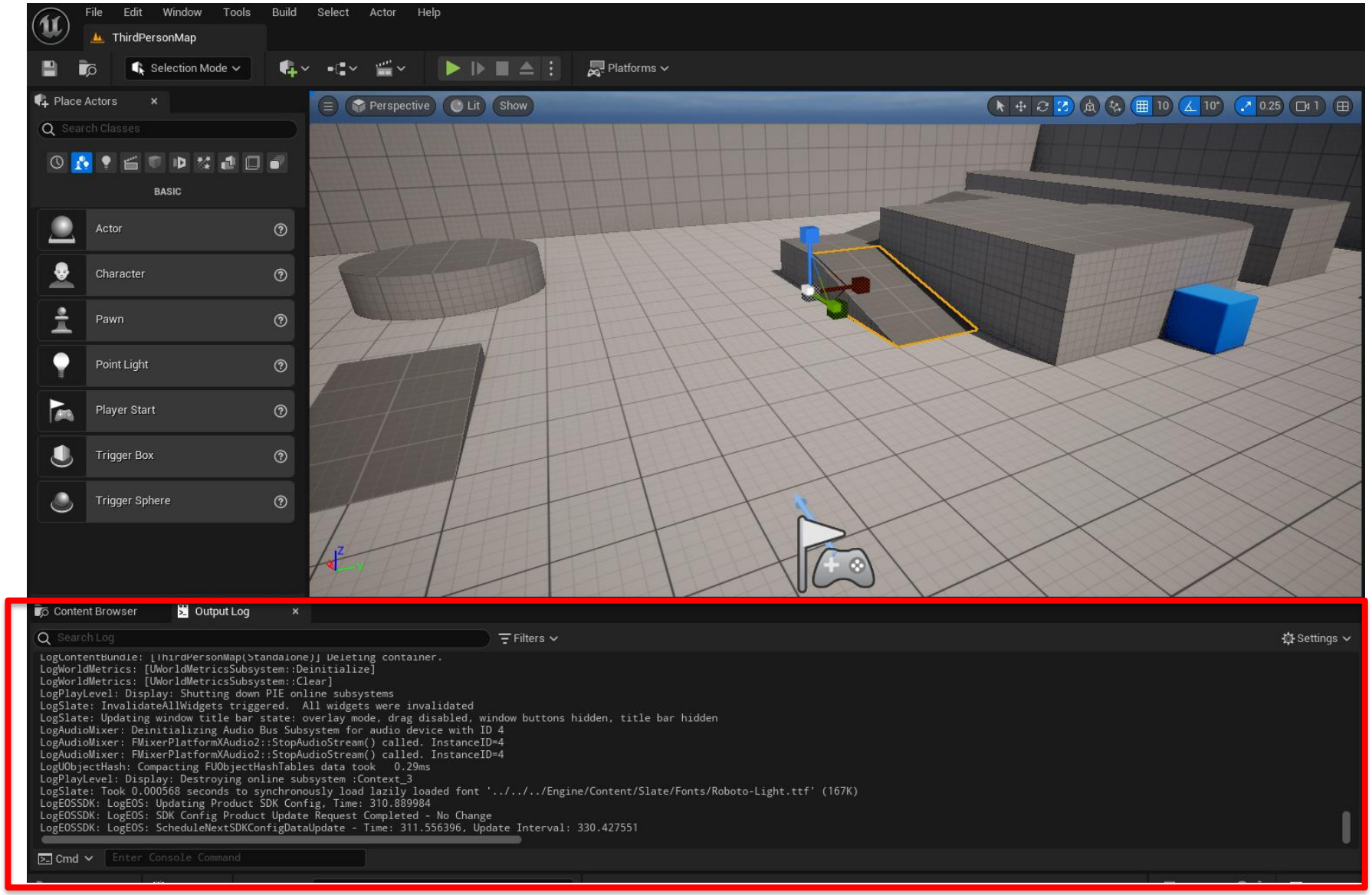
Viewport
(오브젝트를
배치하고, 확인)

Outliner(배치된
오브젝트들의
목록을 확인)

콘텐츠
브라우저
(블루프린트
C++ 파일 등
에셋들을 확인)

디테일
(오브젝트
들의
속성 확인

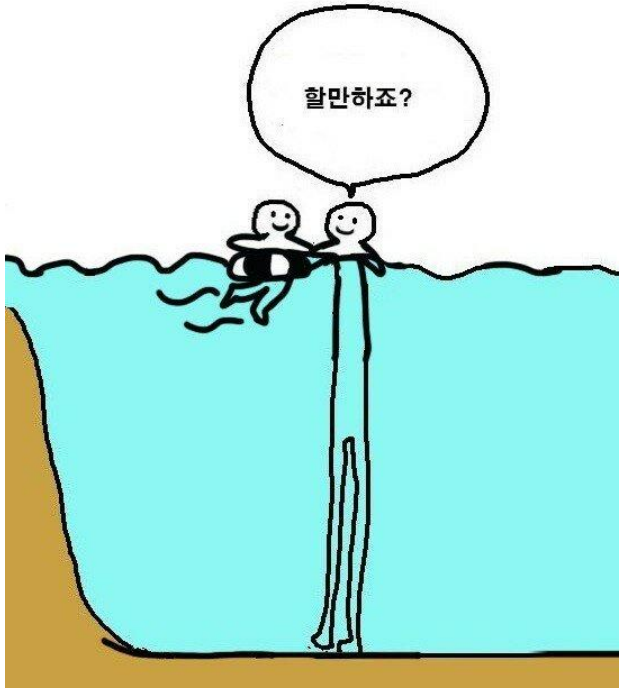
에디터 레이아웃



아웃풋 로그
(가장 중요!
우리가 찍은
로그가
다 여기에 뜸)

실습 0 — 에디터에서 움직여보기

여러분들을 환영합니다.



1. PlayerController

입력을 받아 폰을 조종해보자.

문제 : 캐릭터는 영원하지 않다.

```
void AWeek3JustPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);

    if (UEnhancedInputComponent* EIC = Cast<UEnhancedInputComponent>(PlayerInputComponent))
    {
        EIC->BindAction(MoveAction, ETriggerEvent::Triggered, this, &AWeek3JustPawn::Move);
    }
}

void AWeek3JustPawn::Move(const FInputActionValue& Value)
{
    FVector2D Input = Value.Get<FVector2D>();
    AddActorWorldOffset(FVector(Input.Y, Input.X, 0.f) * MoveSpeed * GetWorld()->GetDeltaSeconds());
}
```

- **3주차 기준: 캐릭터 내부에 입력 처리 코드**
  - 그러나 캐릭터는 죽거나 파괴될 수 있다
  - 캐릭터가 죽으면 입력도 함께 사라짐
  - 캐릭터는 교체될 수도 있다

해결책 1. 그냥 매번 복붙하자.

- 저 많은 코드를 캐릭터마다 다 복붙하자고?

해결책 2. 이런 것들을 모아놓을 클래스를 하나 만들자.

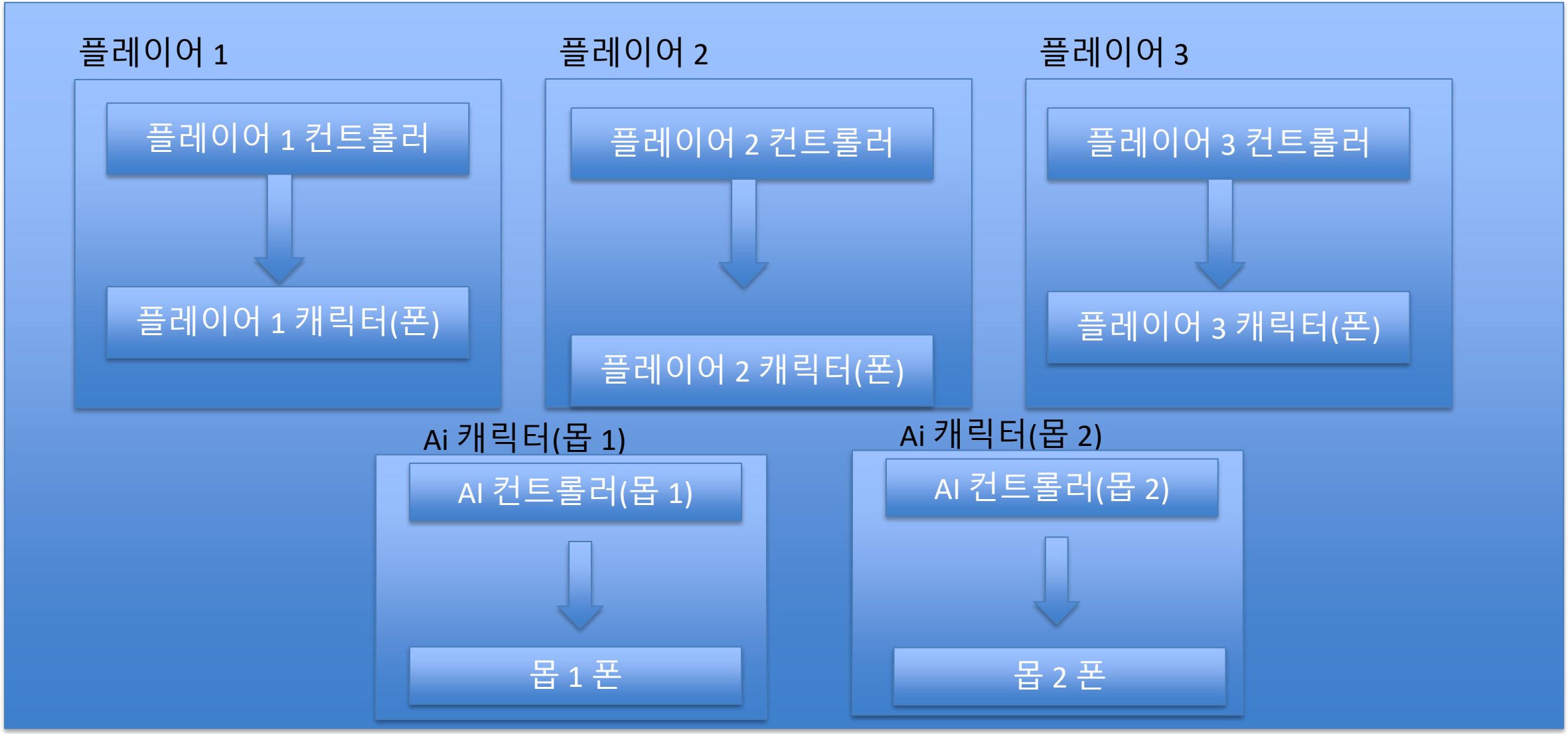
캐릭터 컨트롤 클래스
(컨트롤과 관련된 코드들)

캐릭터
(모델링 정보)
(애니메이션 등..)

PlayerController — 핵심 개념

- **플레이어 컨트롤러 ↔ 플레이어 캐릭터 (단일 연결)**
  - 컨트롤러는 여러 개의 캐릭터를 동시에 연결하지 못한다
  - *컨트롤러 하나로 여러 캐릭터를 조작하는 상황은 흔치 않다*
- **플레이어 캐릭터보다 먼저 생성됨**
  - 플레이어 컨트롤러는 있지만, 플레이어 캐릭터는 없는 상황이 가능하다

PlayerController — 핵심 개념



PlayerController — 핵심 개념

- **Possess(APawn*) — 폰에 빙의**
  - 폰에 실제로 입력이 가도록 연결한다
  - UnPossess() — 폰 해제
- **플레이어 컨트롤러는 폰을 소유하고 있는 것으로 취급**
  - getPawn() → 본인이 소유하고 있는 폰 반환
  - (중요) 폰은 컨트롤러 하나당 하나밖에 소유할 수 없다
- **입력 처리: SetupInputComponent 또는 Enhanced Input 바인딩**
- **UI 관리: CreateWidget + AddToViewport를 호출하는 정석적인 위치**
  - 캐릭터는 죽더라도 UI는 남아야 하는 상황이 많기 때문

Possess – 과정

- GameMode::SpawnDefaultPawnFor() → Pawn 스폰
- GameMode::RestartPlayer() → Possess 호출

APlayerController::Possess(pawn)

- Pawn::PossessedBy(controller) 호출
- OnPossess 이벤트 발생 (BP에서 오버라이드 가능)

- 이후 PlayerController가 입력을 해당 Pawn으로 전달

핵심! PlayerCharacter != Player

그러나 – 조종하고자 하는 의지가 있다면
- 플레이어는 여전히 게임 속에 존재하는 것!

플레이어 캐릭터는 언제든 월드에서 사라질 수 있다.

핵심 규칙 — One Controller, One Pawn

- **PlayerController는 한 번에 하나의 Pawn만 Possess할 수 있다**
  - 멀티플레이: 각 플레이어마다 별도의 PlayerController 인스턴스
  - AI: AIController가 같은 역할 수행 (PlayerController 사용 안 함)
  - Possess 교체 시 이전 Pawn은 자동 UnPossess
- **플레이어 컨트롤러는 Logout 전까지 사라지지 않는다**
  - *즉, 플레이어가 존재한다 → 플레이어 컨트롤러가 존재한다*
- **▶ 오늘 실습에서는 BP_ThirdPersonCharacter를 그대로 사용**

실습 0 — Possess

캐릭터는 바꿀 수 있지만, 그걸 관장하는 주체는 플레이어 컨트롤러다

실습 0: Possess 체험

Step 1 맵 준비 — 캐릭터 A·B 배치

- BP_Gamemode 만들기
- Content Browser → BP_ThirdPersonCharacter 두 개 드래그 → 레벨에 배치
- World Outliner에서 이름 변경: CharacterA, CharacterB
- World Settings → Game Mode Override → BP_Gamemode 선택
- BP_Gamemode 열기 → Default Pawn Class: None 으로 변경
- 저장 (Ctrl+S)

★ DefaultPawnClass=None 이어야 게임 시작 시 자동으로 폰이 스폰되지 않습니다.

실습 0: Possess 체험

Step 2 AWeek3PossessSelector C++ 클래스 생성

- 레벨에 BP_Week3PossessSelector 배치
- 디테일 패널 → CharacterToControl → CharacterA 선택

★ EditAnywhere 이므로 레벨의 디테일 패널에서 폰을 자유롭게 교체할 수 있습니다.

실습 0 체크포인트

- ✅ PIE 시작 시 CharacterToControl에 지정한 폰으로 빙의된다
- ✅ 디테일 패널에서 CharacterB로 바꾸면 다음 PIE에 B로 빙의된다
- ✅ 폰을 정해두고 거기에 플레이어를 빙의시키는 게 아니라,

플레이어 컨트롤러가 정해지고 거기에 빙의시킬 폰을 고르는 것!

▶ **이 PlayerController를 실습 1(스프린트) · 2(HP 바) 의 기반으로 확장한다**

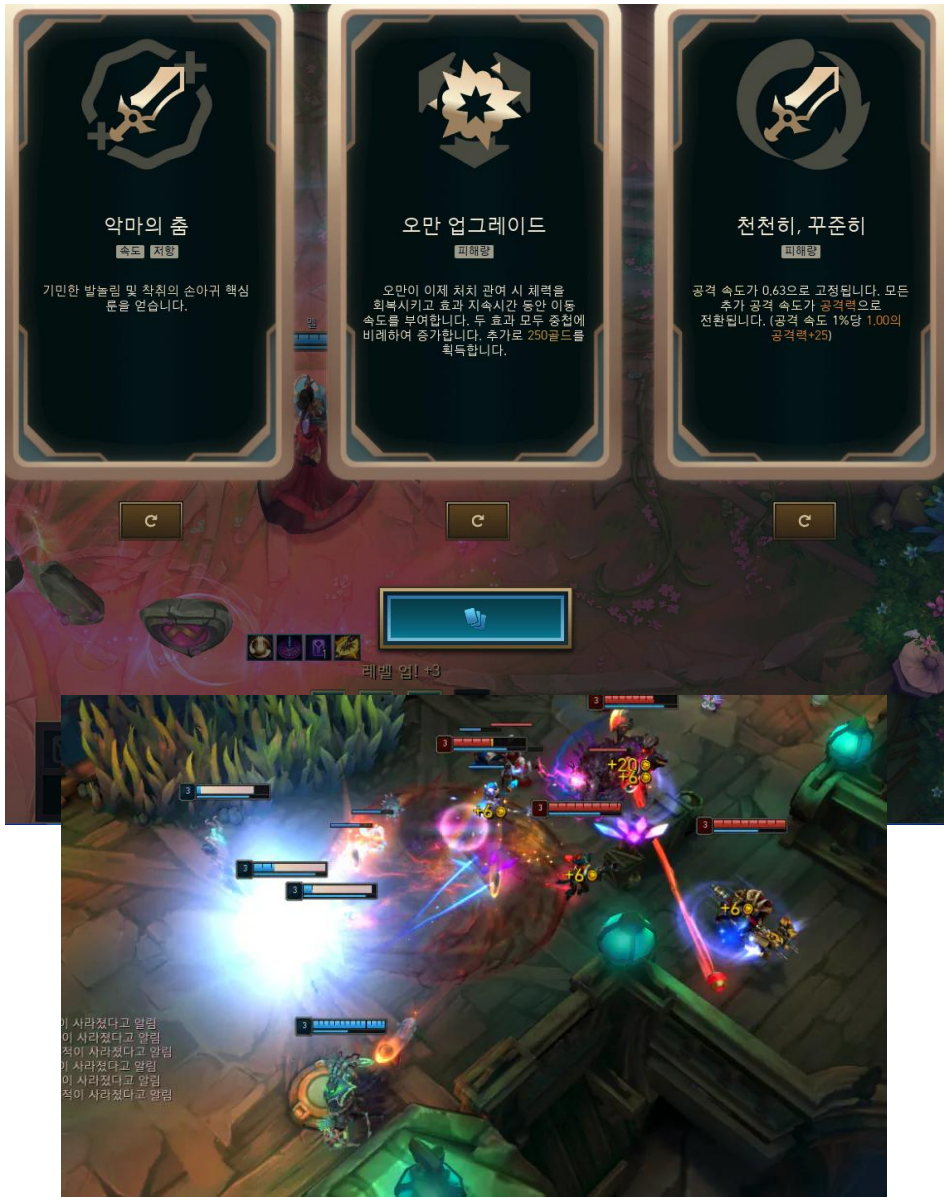
이론 B — PlayerState

플레이어 데이터의 중앙 저장소

PlayerState — 개념

- **APlayerState — 플레이어 고유 데이터 저장**
  - HP, 점수, 이름, 레벨, 경험치 등
- **PlayerController · Pawn 모두에서 접근 가능**
  - GetPlayerState<AMyPlayerState>()
- **Pawn이 파괴되어도 PlayerState는 살아있음**
- **멀티플레이: 서버에서 복제(Replicate)되어 모든 클라이언트에 동기화**
- **오늘 목표: BP_PlayerState에 HP 변수 추가 → 데미지 처리**

캐릭터가 사라져도 남아야 하는 정보?



- 선택한 증강 → 천천히, 꾸준히
- 구매한 아이템: 톱날 단검, 충전형 물약
- 현재 레벨: 3레벨
- **현재 스탯: 공격력 50, 체력 616...**

→ 플레이어 캐릭터에 저장해두자!

캐릭터가 사라져도 남아있어야 하는 정보?

플레이어 캐릭터 삭제함
(메모리 관리)

리스폰 (플레이어 캐릭터 다시
만들어서 플레이어 컨트롤러에 붙이기)

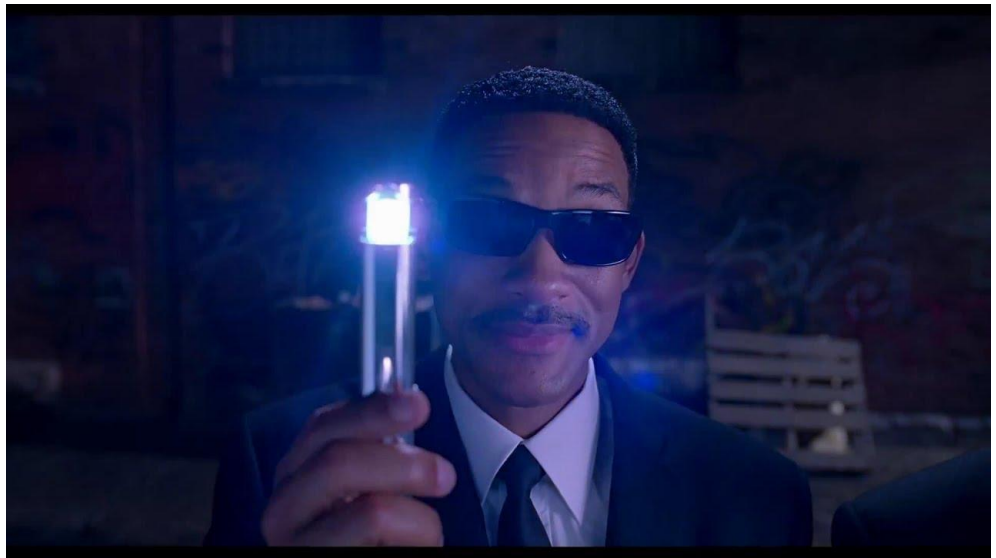
캐릭터가 사라져도 남아있어야 하는 정보?

플레이어 캐릭터 삭제함
(메모리 관리)

리스폰 (플레이어 캐릭터 다시
만들어서 플레이어 컨트롤러에 붙이기)

= 정보가 다 날아가있음!

(플레이어 캐릭터 클래스에 선언된
기본값 그대로)



캐릭터가 사라져도 남아야 하는 정보 = PlayerState

- **플레이어 고유 데이터 저장**
  - HP, 점수, 이름, 레벨, 경험치 등
- **PlayerController · Pawn 모두에서 접근 가능**
  - GetPlayerState<AMyPlayerState>()
- **Pawn이 파괴되어도 PlayerState는 살아있음**
- **멀티플레이: 서버에서 복제(Replicate)되어 모든 클라이언트에 동기화**
- **오늘 목표: BP_PlayerState에 HP 변수 확인**

GameState vs PlayerState

GameState	PlayerState
• 게임 전체 데이터 • 웨이브 번호, 남은 시간 • 모든 플레이어 공유. • GameMode가 관리	• 개별 플레이어 데이터 • HP, 점수, 킬 수 • 해당 플레이어 전용 • PlayerController가 소유
• AGameStateBase 상속	• APlayerState 상속

실습 — PlayerState 설계

- **BP_PlayerState 변수**
  - MaxHP (Float) = 100.0
  - CurrentHP (Float) = 100.0
- **함수 (앞으로 관리할)**
  - TakeDamage(amount) — CurrentHP 감소 + 0으로 클램프
  - IsAlive() — CurrentHP > 0 반환
- *향후 추가: 경험치 / 레벨 / 아이템 보유 목록*

이론 C — Enhanced Input 간략하게

InputMappingContext · InputAction · 런타임 교체

레거시 Input 시스템의 한계

- Project Settings → Input에 Axis/Action 하드코딩
  - 모든 캐릭터가 같은 입력 설정을 공유
- 런타임 키 리맵핑 불가
- 입력 우선순위 제어 어려움
- Modifier(감도 조정 등) 표준 방법 없음

Enhanced Input — 3가지 핵심 에셋

- InputAction (IA_*) — '무엇'을 하는지
  - IA_Move, IA_Jump, IA_Sprint, IA_DebugDamage 등
- InputMappingContext (IMC_*) — 키와 액션 매핑
  - IMC_Default: WASD → IA_Move, Space → IA_Jump
  - IMC_Vehicle: 별도 키 레이아웃
- InputModifier — 값 변환 (Negate, Swizzle, DeadZone 등)
  - Swizzle YXZ: Y축 입력을 X로 → 전후 이동에 사용

런타임 IMC 추가/제거

- BeginPlay 또는 Possess 시점에 IMC 등록
- C++ 코드:
  - UEnhancedInputLocalPlayerSubsystem* Sub =
  - ULocalPlayer::GetSubsystem<...>(PlayerController);
  - Sub->AddMappingContext(IMC_Default, Priority=0);
- Priority가 높은 IMC가 같은 키를 먼저 처리
  - 예: 탈것 탑승 시 IMC_Vehicle(Priority=1) 추가
- RemoveMappingContext(IMC)로 해제 가능

IA 바인딩 패턴 (C++)

- SetupPlayerInputComponent에서:
  - UEnhancedInputComponent* EIC = Cast<UEnhancedInputComponent>(InputComponent);
- Started 트리거 — 키를 처음 누르는 순간 1회
  - EIC->BindAction(IA_Jump, ETriggerEvent::Started, this, &AMyChar::Jump);
- Triggered 트리거 — 키를 누르는 동안 매 프레임
  - EIC->BindAction(IA_Move, ETriggerEvent::Triggered, this, &AMyChar::Move);
- Completed 트리거 — 키를 뗄 때 1회
  - EIC->BindAction(IA_Sprint, ETriggerEvent::Completed, this, &AMyChar::StopSprint);

실습 1 — 스프린트 구현

IA_Sprint + MaxWalkSpeed 변경 (15분)

실습 1: 스프린트

Step 1 IA_Sprint InputAction 에셋 생성

- Content Browser → 우클릭 → Input → Input Action
- 이름: IA_Sprint
- Value Type: Digital (bool)
- 저장 (Ctrl+S)

★ *Value Type을 'Digital'로 설정하지 않으면 Triggered가 매 프레임 호출됩니다.*

실습 1: 스프린트

Step 2 IMC_Default에 Shift 키 매핑

- IMC_Default 열기 → Mappings + 버튼
- Action: IA_Sprint 선택
- 키: Left Shift
- 저장

실습 1: 스프린트

Step 3 Week3Character.h 에 함수·변수 확인

- UPROPERTY(EditAnywhere) float SprintSpeed = 800.f;
- UPROPERTY(EditAnywhere) float WalkSpeed = 500.f;
  - (기본값은 CharacterMovement의 MaxWalkSpeed와 맞추세요)
- void Sprint(const FInputActionValue& Value);
- void StopSprint(const FInputActionValue& Value);

실습 1: 스프린트

Step 4 Week3Character.cpp 구현 + 바인딩

- Week3Character.cpp — 함수 본문과 바인딩을 채워 넣는다 ← *직접 작성*

```
void AWeek3Character::Sprint(const FInputActionValue&)
{
    GetCharacterMovement()->MaxWalkSpeed = SprintSpeed;   ← 작성
}
void AWeek3Character::StopSprint(const FInputActionValue&)
{
    GetCharacterMovement()->MaxWalkSpeed = WalkSpeed;   ← 작성
}
```

- SetupPlayerInputComponent에 BindAction 두 줄 추가 ← *작성*
  - `EIC->BindAction(SprintAction, ETriggerEvent::Started,   this, &Sprint);`
  - `EIC->BindAction(SprintAction, ETriggerEvent::Completed, this, &StopSprint);`
- SprintAction 변수(UPROPERTY)에 IA_Sprint 에셋 연결 (BP에서)

★ *빌드 후 BP_ThirdPersonCharacter 디테일 패널에서 SprintAction 슬롯이 보여야 합니다.*

실습 1 체크포인트

- ✅ Shift 누르는 동안 이동 속도가 빨라진다
- ✅ Shift를 떼면 원래 속도로 돌아온다
- ✅ BP_ThirdPersonCharacter 디테일에서 SprintSpeed 값 변경 가능
- ✅ Output Log에 에러 없음

실습 2 — HP 바 UMG 위젯

WBP_PlayerHUD 생성 + 화면에 표시 (20분)

실습 2: HP 바 위젯

Step 1 WBP_PlayerHUD 위젯 블루프린트 생성

- Content Browser → 우클릭 → User Interface → Widget Blueprint
- 이름: WBP_PlayerHUD
- 기본 클래스: UserWidget
- 더블 클릭 → Widget Designer 열기

실습 2: HP 바 위젯

Step 2 UI 레이아웃 구성

- Palette → Canvas Panel 추가 (루트)
- 좌하단에 Vertical Box 배치
- Vertical Box 안에 Text Block 추가: 'HP'
- Text Block 아래에 Progress Bar 추가
  - Size: 200×20, Fill Color: 빨간색(R=1,G=0,B=0)
- 바인딩 추가

★ *'Is Variable'을 체크해야 그래프에서 HPBar 변수에 접근할 수 있습니다.*

실습 2: HP 바 위젯

Step 3 Percent 바인딩 + PlayerController에서 위젯 생성

※ 실습 0에서 만든 Week3PlayerController를 확장합니다

- HPBar → Percent → Bind → Create Binding
- 바인딩 함수 안: Get Player State → Cast to BP_PlayerState
  - → CurrentHP / MaxHP → Return (Float)
- BP_PlayerController -> DefaultWidget 변경

★ *Cast 실패 시 분기를 잊으면 게임이 크래시됩니다. IsValid로 항상 확인!*

실습 2 체크포인트

- ✅ 게임 시작 시 좌하단에 빨간 프로그레스 바가 보인다
- ✅ 바 Percent가 1.0 (100%)으로 꽉 차 있다
- ✅ 위젯이 PIE 종료 후 다시 시작해도 표시된다
- 아직 HP가 고정값 — 실습 3에서 데미지를 연결합니다

실습 3 — 1초마다 데미지 + 죽으면 리스폰

OnTickDamage · RespawnCharacter (20분)

실습 3: 1초마다 데미지 + 리스폰

Step 1 시나리오 이해 — 흐름 그리기

- **이번 실습은 두 군데에 코드를 채워 넣는다**
  - `Week3Character::OnTickDamage` — 1초마다 PlayerState에 데미지 1 적용
  - `Week3PlayerController::RespawnCharacter` — 새 폰을 Possess하고 HP 리셋
- **PlayerState는 이미 준비되어 있다** *(건드리지 않는다)*
  - BeginPlay에서 CurrentHP = MaxHP 초기화
  - ApplyDamage(amount), IsAlive(), ResetHP() 함수 모두 구현됨
- **전체 흐름:** `Tick(1초) → ApplyDamage(1) → HP=0 → IsAlive=false → Destroy → ScheduleRespawn → 0.5초 → Possess + ResetHP`

실습 3: 1초마다 데미지 + 리스폰

Step 2 Character::OnTickDamage 작성

- **Week3Character.cpp — OnTickDamage() 함수 안에 한 줄 추가** *← 직접 작성*

```
void AWeek3Character::OnTickDamage()
{
    AWeek3PlayerState* PS = GetPlayerState<AWeek3PlayerState>();
    if (!PS) return;
    PS->ApplyDamage(1.f);   ← 여기!
    if (!PS->IsAlive()) { /* Destroy + ScheduleRespawn */ }
}
```

- BeginPlay에서 `SetTimer(... 1.0f, true)`로 1초마다 자동 호출됨

★ *ApplyDamage 한 줄만 작성하면 됩니다 — 나머지는 이미 PlayerState에 있어요.*

실습 3: 1초마다 데미지 + 리스폰

Step 3 PlayerController::RespawnCharacter 작성

- Week3PlayerController.cpp — RespawnCharacter() 안의 if 블록 채우기 ← *직접 작성*

```
void AWeek3PlayerController::RespawnCharacter()
{
    if (!LastPawnClass) return;
    APawn* NewPawn = GetWorld()->SpawnActor<APawn>(LastPawnClass, ...);
    if (NewPawn)   ← 여기 안쪽
    {
        Possess(NewPawn);
        if (AWeek3PlayerState* PS = GetPlayerState<AWeek3PlayerState>())
            PS->ResetHP();
    }
}
```

실습 3: 1초마다 데미지 + 리스폰

Step 4 빌드 · 테스트 · 동작 확인

- **에디터 빌드 (Ctrl+Alt+F11 또는 빌드 버튼)**
- **PIE 실행 → 캐릭터 가만히 두기**
  - HP 바가 1초마다 1씩 줄어드는지 확인 (1% 씩)
- **100초 후 HP가 0이 되면...**
  - 캐릭터가 사라지고 (Destroy)
  - 0.5초 뒤 같은 위치에 새 캐릭터가 스폰되어 다시 조작 가능
  - HP 바가 100%로 다시 차오른다 (ResetHP)
- **Output Log: 에러 없음 확인**

★ *리스폰이 안 되면: PlayerController가 BP_Week3PlayerController인지 World Settings 확인.*

실습 3: 1초마다 데미지 + 리스폰

Step 5 전체 데이터 흐름 다이어그램

- **1초마다 데미지 — 흐름**

`Timer(1s) → Character::OnTickDamage → PlayerState::ApplyDamage(1) → UI(Percent Binding)`

- **죽으면 리스폰 — 흐름**

`HP=0 → IsAlive=false → Character::Destroy + PC::ScheduleRespawn`

`0.5s Timer → RespawnCharacter → SpawnActor + Possess + ResetHP`

- **핵심:** PlayerState는 Pawn이 사라져도 살아남는다 → HP/스탯 영속
  - 그래서 **`ResetHP`**를 명시적으로 호출해야 다시 100이 된다

★ *Possess만 하고 ResetHP를 빼먹으면 — 새 캐릭터가 즉시 또 죽는다 (HP가 이미 0이라서).*

실습 3 체크포인트 — 데미지 + 리스폰 완성

- ✅ 1초마다 HP 바가 1씩 줄어든다
- ✅ HP=0이 되면 캐릭터가 사라지고, 0.5초 뒤 같은 자리에 다시 스폰된다
- ✅ 리스폰 후 HP가 100으로 리셋되어 화면에 다시 채워진다
- ✅ *전체 흐름: Tick → Character → PlayerState → (UI / IsAlive) → PC::Respawn*

▶ 다음 주: 이 기반 위에 적 스폰 + 피격 이벤트 추가

정리 & Q&A

오늘 배운 것

- **실습 0: PlayerController::Possess()로 원하는 폰에 직접 빙의**
  - *[기반] PlayerController — Possess()로 빙의, 입력·UI·빙의 담당*
- **PlayerState — 플레이어 데이터 중앙 저장소, 폰이 파괴되어도 살아남음**
- **Enhanced Input — IA + IMC 분리로 유연한 키 리매핑**
- **UMG ProgressBar + Percent Binding으로 실시간 HP 바**
- **실습 3:** `Tick 데미지 → IsAlive 체크 → Destroy → Respawn(Possess + ResetHP)`
- **전체 흐름: `Input → Controller → Pawn(Character) → PlayerState → UI`**